Today's Session

Part 1: Orientation 2:00–3:00 PM

- Course overview, structure & rules
- The Big Data landscape in 2026
- Our complete tool stack
- 17-week journey overview
- Assessment, project & expectations

Part 2: Hands-On Setup 3:10–4:30 PM

- Hardware & software check
- Docker Desktop installation
- Launching the Big Data stack
- First PySpark job (live coding)
- Exploring the Spark Web UI

Take-Home Assignment

Due Sunday 15 March, 23:59

Complete full environment setup + run the getting-started notebook + submit screenshots and a 100-word reflection on Google Classroom.

Course Snapshot

Course No.	Z5008
Title	Big Data Lab
Credits	6 (3 hrs contact + 6–9 hrs self-work/week)
Slot	Monday 2:00–4:40 PM (Slot P)
For	MTech DS&AI - Semester 2 (Core)

Prerequisites:

- Python programming (intermediate level)
- Basic data structures & databases
- Laptop with $\geq$8 GB RAM & 20 GB free disk

Assessment at a Glance

Written Quiz (Week 7)	15%
Mid-Sem Lab Exam (Week 13)	15%
End-Sem Lab Exam	20%
Course Project	**50%**
Total	**100%**

The **project** is 50% of your grade. Start planning your domain from **Day 1**. Individual.

Why Big Data Matters in 2026

The Scale of the Problem

- **4.6 quintillion bytes** generated every single day
- **75+ billion** IoT devices active worldwide
- LLMs trained on **petabytes** of curated text
- Real-time AI demands **sub-second** decisions
- 90% of all data created **in the last 2 years**

The Career Opportunity

- Data Engineers earn are well paid
- 3 million+ open data roles globally (2025)
- Every company is now a data company

Real-World Applications

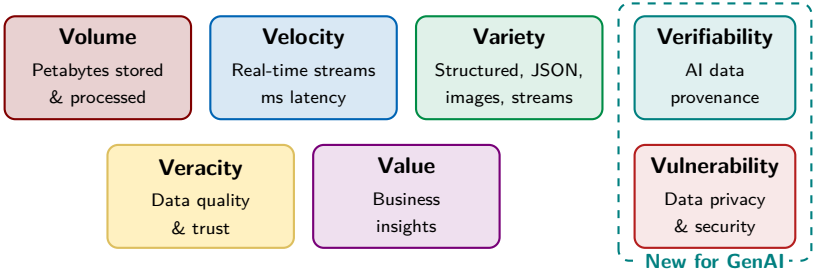
- **Healthcare:** Genomic pipelines, clinical AI
- **Agriculture:** Precision farming, sensors
- **Finance:** Fraud detection ($<$50 ms latency)
- **Retail:** Real-time personalisation
- **Transport:** Fleet & logistics optimisation
- **Climate:** Petabyte-scale weather models

The African Context

Mobile money, precision agriculture, health data pipelines - Big Data is **Africa's next competitive frontier**.

This is your edge.

The 5 V's of Big Data (+ 2 New Ones for 2026)



Why Verifiability & Vulnerability?

In the LLM era, knowing *where* your data came from (Provenance) and ensuring it doesn't leak personal information (Privacy) are no longer afterthoughts, they dictate your entire pipeline architecture.

DO.jpg

The Modern Big Data Stack (2026)

SERVING & MONITORING
BentoML · Grafana · Prometheus · REST APIs

ML & MLOps
Spark MLlib · Ray · MLflow · Great Expectations

PROCESSING & ORCHESTRATION
Apache Spark 3.5+ · Airflow · Dagster · dbt

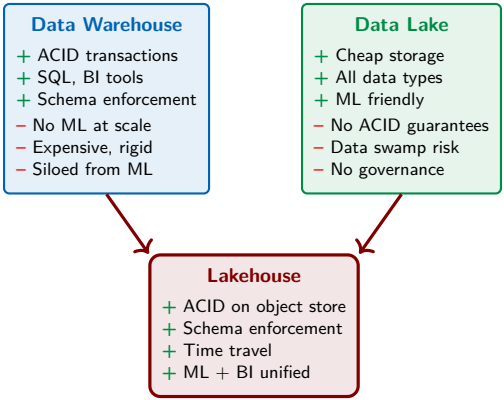
LAKEHOUSE FORMAT
Delta Lake · Apache Iceberg · Schema Evolution · Time Travel

OBJECT STORAGE
MinIO (S3-compatible) · Parquet · ORC

INGESTION
Kafka 3.7+ · Redpanda · REST APIs · CDC

We will build proficiency at every layer of this stack over our 17-week journey.

The Lakehouse Paradigm



Key Technologies

- **Delta Lake** — Databricks
- **Apache Iceberg** — Netflix/Apple
- **Apache Hudi** — Uber

This Course

We use **Delta Lake** and **Iceberg** on **MinIO** (S3-compatible local storage) throughout the semester.

DO.jpg

Your Technology Stack for This Course

Category	Tool(s)
Language	Python 3.11+ (primary)
Big Data Engine	Apache Spark 3.5+
Lakehouse Format	Delta Lake, Iceberg
Object Storage	MinIO (S3-compatible)
Streaming	Apache Kafka 3.7+
Orchestration	Airflow 2.9, Dagster
ML / Features	Spark MLlib, Ray
MLOps	MLflow, Great Expectations
Deployment	BentoML, Docker
Monitoring	Grafana, Prometheus
IDE	JupyterLab, VS Code
Cloud (backup)	Databricks Community Ed.

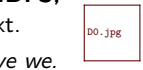
Why These Tools?

- **Industry standard** — Netflix, Spotify, Grab
- **Open source** — free forever, career-portable
- **2026 relevant** — Lakehouse + MLOps is the market
- **Docker-based** — run the stack locally
- **Cloud-portable** — same code on AWS/GCP/Azure

Deprecated Context

We will only briefly cover **Hadoop, HDFS, and MapReduce** for historical context.

The industry has moved on — so have we.



Your 17-Week Journey

FOUNDATIONS	EXAMS	STREAMING	ML & MLOps	PROJECT
Weeks 1–7	W. 8	W. 9–10	Weeks 11–15	W. 16–17

W	Topic
1	Course orientation + environment setup
2	Lakehouse architecture + Delta Lake intro
3	Delta Lake advanced + Apache Iceberg
4	Spark 3.5+ foundations & DataFrame API
5	Spark SQL + joins + window functions
6	Spark performance tuning + AQE
7	Orchestration: Airflow + Dagster
8	**MID-SEMESTER: Written Quiz + Lab**

W	Topic
9	Real-time streaming: Kafka + Redpanda
10	Spark Structured Streaming + stateful
11	Visualisation: Grafana + Streamlit
12	Distributed ML: MLlib + Ray
13	MLOps: MLflow tracking + registry
14	Model deployment: BentoML + Docker
15	Monitoring + scaling + data governance
16	Project integration + load testing
17	**FINAL PROJECT PRESENTATIONS**

Assessment in Detail

Grading Distribution

Component	Weight
Written Quiz (Week 8)	15%
Mid-Sem Lab Exam (Week 13)	15%
End-Sem Lab Exam (W. 17+)	20%
Course Project	**50%**
Total	**100%**

Project (50%) Breakdown

Deliverable	Weight
Proposal + Architecture	10%
Progress Reviews ($\times 2$)	10%
GitHub Repo Quality	10%
Final Demo + Load Test	10%
Presentation + 8-min Video	10%

Lab Exam Format

- Individual work; open-notes allowed
- Pre-configured Docker environment
- Tasks: Write Spark jobs, debug pipelines, query tables
- Graded on: Correctness, efficiency, style

Academic Integrity

AI coding tools (Copilot, ChatGPT) are **allowed** but **must be declared**. You must be able to explain every line of code during Q&A. Identical submissions will receive a zero.

The Course Project: What You Will Build

Real-Time Lakehouse + MLOps Intelligence Platform

Build a production-grade, end-to-end data system **individually or in pairs (max 2)**.
Choose any domain: agriculture, health, finance, transport, environment, retail...

Required Components:

1. Streaming ingestion pipeline (Kafka)
2. Lakehouse storage layer (Delta Lake/Iceberg)
3. Batch + streaming processing (Spark)
4. Orchestrated workflows (Airflow/Dagster)
5. ML model trained at scale (MLlib/Ray)
6. MLflow: experiment tracking + registry
7. Deployed model API (BentoML + Docker)
8. Live monitoring dashboard (Grafana)

Milestone Timeline:

- **Week 2:** Project brief released
- **Week 3:** Partner selection & domain choice
- **Week 4:** Proposal + architecture diagram
- **Week 8:** Progress Review 1 (10m demo)
- **Week 12:** Progress Review 2 (arch. check)
- **Week 15:** Code freeze
- **Week 16:** Load test + dress rehearsal
- **Week 17:** Final presentation + live demo

Bonus (+5%): Add a natural-language query interface using an LLM e.g., GPT-4o, Gemini, local Llama over your data pipeline.

Lab Policies & Expectations

Attendance & Participation

- $\geq$85% attendance is **mandatory**
- Missing a lab = missed irreplaceable practice
- Arrive on time; >15 min late = absent
- **Bring your charged laptop every session**

During Lab Sessions

- Focused work; no social media/gaming
- **Collaboration encouraged** - help neighbours
- Ask questions loudly - no silly questions
- Document your errors; they are your best teacher

Weekly Assignments

- Not graded - but **exam questions come from them**
- Complete *before* the next session
- Discussed at the start of each lab (15 min)
- Submit on Moodle for feedback

Communication

- Email: `innocent@iitmz.ac.in`
- Response time: within 24 h on weekdays
- Google Classroom: for urgent setup questions
- Office hours: by appointment (email first)
- Stack Overflow & docs: **use them first**

10-Minute Break

Back at **3:10 PM** for hands-on setup

Stretch, hydrate, use the bathroom

"In God we trust. All others must bring data."

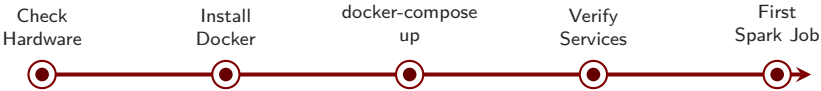
— W. Edwards Deming

Our Goal for the Next 80 Minutes

Have a fully working Big Data development environment on your laptop:

Apache Spark 3.5 · **Delta Lake** · **Apache Kafka** · **MinIO** · **MLflow**

all running locally via a single `docker-compose up` command.



See the printed `setup_guide.pdf` handout in Google Classroom.

Hardware & Software Requirements

Hardware Requirements

- CPU: 64-bit (Intel/AMD/Apple Silicon)
- RAM: **8 GB minimum** (16 GB rec.)
- Disk: **20 GB free** (for Docker images)
- OS: Windows 10/11, macOS 12+, Ubuntu

Apple Silicon (M1/M2/M3)?

Docker Desktop supports ARM64. All images in our stack have ARM64 variants. If an older image fails, you may need:

```
platform: linux/amd64
```

Software to Install

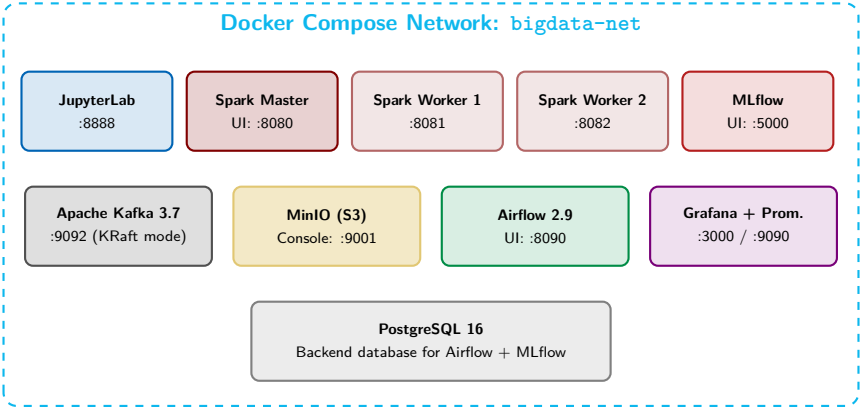
1. **Docker Desktop** (free)
   `docker.com/products/docker-desktop`
2. **VS Code** + Python extension
   `code.visualstudio.com`
3. **Git** (for course materials)
   `git-scm.com`

Laptop Struggling? Backup Options:

Use **Databricks Community Edition** (free cloud): `community.cloud.databricks.com`. Sign up with your `iitmz.ac.in` email.

Google Colab also supports PySpark!

Our Docker Compose Big Data Stack



Warning: Total RAM usage is ≈6–8 GB when all services are running.
Close other heavy applications (like Chrome) during the lab.

DO.jpg

Step 1: Launch the Stack

Open Git Bash (Windows) / Terminal (Mac/Linux):

```
# Create your working directory
mkdir ~/bigdata-lab && cd ~/bigdata-lab

# Copy docker-compose.yml from USB or download from Moodle
# (place it inside ~/bigdata-lab/)

# Copy the environment file
cp .env.example .env

# FIRST-TIME: Pull all Docker images (~8 GB - use USB if WiFi is slow)
docker-compose pull

# Start all services in background (-d = detached mode)
docker-compose up -d

# Verify all containers are running
docker-compose ps
# You should see: State = "running" for all 9 services
```

Using the USB Drive (Faster / No Internet Needed)

Load the pre-cached images directly from the provided USB drive to skip the 8 GB download:

```
docker load -i /path/to/usb/bigdata-images.tar.gz
```

Step 2: Verify All 8 Services

Service	URL	Login	What to Check
JupyterLab	`localhost:8888`	token: `bigdata`	Open a notebook
Spark Master UI	`localhost:8080`	none	2 workers listed
MLflow UI	`localhost:5000`	none	Experiments page
MinIO Console	`localhost:9001`	admin / bigdata123	Dashboard loads
Airflow	`localhost:8090`	admin / admin	DAGs page
Grafana	`localhost:3000`	admin / admin	Home page
Prometheus	`localhost:9090`	none	Targets page
Kafka	`localhost:9092`	—	Test via Python

Something Failed? Quick Debug Steps

1. Check logs: `docker-compose logs <service>`
2. Check RAM: Docker Desktop → 10 GB+
3. Restart: `docker-compose restart`
4. Ask in the class Google Classroom

Your First PySpark Job

Go to `localhost:8888`, open `lab01_getting_started.ipynb`:

```python
from pyspark.sql import SparkSession

# Create SparkSession connected to our cluster
spark = SparkSession.builder \
    .appName("Lab01 - Hello Big Data") \
    .master("spark://spark-master:7077") \
    .config("spark.sql.extensions",
            "io.delta.sql.DeltaSparkSessionExtension") \
    .config("spark.sql.catalog.spark_catalog",
            "org.apache.spark.sql.delta.catalog.DeltaCatalog") \
    .getOrCreate()

print(f"Spark version: {spark.version}")    # Should print: 3.5.x
print(f"Worker nodes: {spark.sparkContext.defaultParallelism}")

# Create your first DataFrame
data = [("Alice", 28, "Nairobi"), ("Bob", 32, "Dar es Salaam"),
        ("Chloe", 25, "Zanzibar"), ("David", 29, "Kampala")]

df = spark.createDataFrame(data, ["name", "age", "city"])
df.show()
df.printSchema()
```

Write Your First Delta Lake Table

```python
import os
# Configure MinIO S3 credentials (already set in docker-compose)
# spark._jsc.hadoopConfiguration().set("fs.s3a.endpoint", "http://minio:9000")

# Write DataFrame as a Delta Lake table to MinIO
df.write \
    .format("delta") \
    .mode("overwrite") \
    .save("s3a://warehouse/lab01/people")

print("Successfully written to Delta Lake on MinIO!")

# Read it back
df_delta = spark.read.format("delta").load("s3a://warehouse/lab01/people")
df_delta.show()

# Explore the Delta transaction log
from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, "s3a://warehouse/lab01/people")
dt.history().show(truncate=False)   # See the full change history!
```

What Just Happened?

You wrote data to an **object store** (MinIO) using the **Delta Lake** format. This is the **core Lakehouse pattern** we will use all semester. Check the MinIO Console (:9001) to visually inspect the

Explore the Spark Web UI

Open `localhost:8080` in your browser:

- **Jobs:** Every DataFrame action becomes a job
- **Stages:** Jobs split into parallel stages
- **Tasks:** Stages split into tasks per partition
- **Storage:** Cached DataFrames/RDDs
- **Executors:** Worker resources and usage
- **SQL/DataFrame:** Visual query plan (Catalyst)
- **Environment:** Spark configuration

Pro Tip:
The Spark UI is your most powerful debugging tool. We dedicate an entire session to reading it deeply in **Week 6 (Performance Tuning)**. Start getting familiar with it now.

Spark UI: What to Look For

- **Task duration** – shuffle-heavy?
- **GC time** – memory pressure?
- **Data spills** – partition too large?
- **Skew** – one task much slower?
- **Stage DAG** – bottlenecks visible

Try It Now

After running your notebook, go to the **SQL/DataFrame** tab and click on the query.

You will see the visual Catalyst query plan!

Common Issues & Fixes

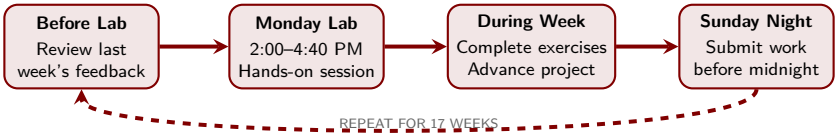
Problem	Solution
`docker pull` fails	Use USB: `docker load -i bigdata-images.tar.gz`
Spark UI: 0 workers	Wait 60 s; check `docker logs spark-worker-1`
JupyterLab blank page	Go to `localhost:8888/lab`, token: `bigdata`
MinIO connection refused	Check `docker-compose ps minio` is *running*
Out of memory (OOM)	Increase Docker RAM to 10 GB (Resources → RAM)
Windows: Docker not found	Ensure WSL2 + Docker Desktop are active
`OutOfMemoryError`	Restart: `docker-compose restart spark-master`
S3 / MinIO "Access Denied"	Check `.env` for `AWS_ACCESS_KEY_ID=admin`

The Reset Option:

```
docker-compose down -v && docker-compose up -d
```

This destroys all containers and volumes (data) to start completely fresh. Use this only if a service is hopelessly corrupted or stuck.

Your Weekly Lab Workflow



Suggested Weekly Time Budget

Activity	Hours	Activity	Hours
Monday Lab Session	3 hrs	Project (Group Work)	2–3 hrs
Notebook Assignments	3–4 hrs	Reading / Videos	1–2 hrs

Total: 10–12 hours per week

Consistency is key in Big Data. Falling behind by two weeks makes catching up very difficult.

DO.jpg

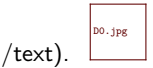
Week 1 Assignment

Due: Sun 15 March, 23:59

Assignment 1: Environment Setup + Exploration

1. **Complete Full Setup:** Submit a screenshot of `docker-compose ps` showing all 9 services in *running* state.
2. **Execute Lab Notebook:** Run all cells in `lab01_getting_started.ipynb`. Add 3 custom DataFrame operations e.g., a new filter or sort.
3. **Exploration Task:** Load any small CSV e.g., from Kaggle:
   - (a) Show basic statistics (`df.describe().show()`)
   - (b) Filter rows by at least one specific condition
   - (c) Group by a column and compute an aggregate (count/avg)
   - (d) Write the final result in **Delta Lake** format to MinIO
4. **Reflection (100 words):** What was the most challenging part of the setup? What are you most excited to learn?

Submission (Google Classroom): Upload `.ipynb` file + `.png` screenshot + Reflection (`.pdf`/text).



Coming Up: Lab 2 - Lakehouse Architecture

Monday 16 March:

- **Deep Dive:** Warehouse vs. Lake vs. Lakehouse
- **Delta Internals:** Transaction logs & checkpointing
- **ACID Operations** on object storage (S3/MinIO)
- **Schema Control:** Enforcement vs. Evolution
- **Hands-on:** MERGE/UPSERT and "Time Travel"

Prepare for Lab 2

- **Read:** *"What is a Lakehouse?"* – Databricks Blog (Link on Moodle)
- **Watch:** *Lakehouse Intro* (15 min)
- **Check:** Ensure your **MinIO** is running!

Project Brief Preview

Released on Google Classroom after Lab 2. Start thinking about your domain: *Agriculture, Health, Finance, or Transport?*

See you in the Lab at 2:00 PM!

Self-Study Expectations (Weekly)

This course cannot be completed from slides alone

- This is the first lecture: these slides are **my summaries**, not the full material.
- We meet only **once per week**, so you must do **significant reading and practice on your own**.
- Use the official **documentation, tutorials, and reference guides** for every tool we touch (Spark, Hadoop, Kafka, Airflow, Docker, etc.).
- Come to lab with questions from your reading and issues you hit while practicing.

Resources: Tools & Recommended Reading

Official docs (start here)

- Apache Spark Documentation
- Apache Kafka Documentation
- Apache Airflow Documentation
- Docker Documentation
- Apache Hadoop Documentation
- Delta Lake (delta.io)
- Apache Iceberg
- MLflow Documentation
- Grafana Documentation

Recommended books / long reads

- Martin Kleppmann: *Designing Data-Intensive Applications*
- Bill Chambers & Matei Zaharia: *Spark: The Definitive Guide*
- Holden Karau et al.: *Learning Spark (2nd ed.)*
- Tyler Akidau et al.: *Streaming Systems*
- Neha Narkhede et al.: *Kafka: The Definitive Guide*

Weekly habit

Read the official docs **while** you build; keep notes of errors, commands, configs, and "gotchas".

Questions?

"The goal of Big Data is to turn data into information, and information into insight."

— Carly Fiorina